InventoryHub

Product Inventory Manager

A Full-Stack C# and .NET Project — From Local Code to Docker, Kubernetes, Azure and AWS

By **Abel Haile Magicho**

GitHub: github.com/kembAB

This explainer walks through the whole project in the order it was built. It shows what the application does, how the pieces fit together, the main code files in full, the real output of a verified local run, and the exact deployment path for Azure and for AWS.

The build and the run in this document are real. The project was compiled with the .NET 8 SDK and exercised with live API calls before this explainer was written. The cloud sections are configuration-ready deployment guides. **No live cloud account was used.** That limit is stated again in the limitations chapter.

Chapter 1 — What the Project Is

InventoryHub manages the product catalogue of a store or a small warehouse. A user opens one web page, sees every product with its price and stock level, adds new products, edits them, corrects stock counts and deletes products. Rows running low on stock are highlighted, and out-of-stock rows are marked in red.

The backend is a **REST API** built with **ASP.NET Core 8** and written in **C#**. The frontend is a single page of HTML, CSS and JavaScript stored in the wwwroot folder. **The API application serves the frontend itself.** There is no separate web server and no second deployment. One build produces one artifact, and that artifact is what runs on a laptop, inside a Docker container, on a Kubernetes cluster, on Azure and on AWS.

Data lives in a real database through **Entity Framework Core**. Locally the database is **SQLite**, a single file that needs no server and no setup. In Docker and in the cloud the same application uses **PostgreSQL**. **The switch between the two databases is configuration only.** A setting named DatabaseProvider chooses the provider at startup, and the connection string comes from configuration or from an environment variable. Not one line of application code changes.

The API surface

- **GET /api/products** — list every product.
- **GET /api/products/{id}** — fetch one product by its id.
- **GET /api/products/low-stock** — list products whose stock is below a threshold. **The threshold is a query parameter and defaults to 5.**
- **POST /api/products** — create a product. The response is HTTP 201 with the created product, including its new id.
- **PUT /api/products/{id}** — replace every editable field of a product.
- **PATCH /api/products/{id}/stock** — change only the stock quantity. **This is the endpoint the stock editor in the frontend uses.**
- **DELETE /api/products/{id}** — delete a product. The response is HTTP 204.

Validation is built into the model. A product must have a name, a SKU and a category. The price must be at least 0.01 and the stock quantity must never be negative. **An invalid request is rejected with HTTP 400 before the controller code runs.** The SKU is also unique in the database, and creating a second product with an existing SKU returns HTTP 409. **SKU uniqueness is enforced by a database index, not only by application code.** That matters when two requests arrive at the same moment.

Chapter 2 — Architecture

The request path has four layers. The browser loads the single-page frontend. JavaScript on the page calls the API with fetch. The **controller** receives the call, validates the input and talks to the **DbContext**. The DbContext translates the C# queries into SQL and runs them against the configured database.

- **ProductsController** — the HTTP layer. It owns the routes, the status codes and nothing else. Business rules that belong to the data itself live in the model and in the database schema.
- **Product** — the entity. Its properties map one to one onto the columns of the Products table, and its data annotation attributes carry the validation rules.
- **AppDbContext** — the Entity Framework Core context. It exposes the Products set, declares the unique SKU index and holds the seed data.
- **Program.cs** — the composition root. It registers the services, reads the DatabaseProvider setting, creates the database on startup, inserts the seed data when the table is empty and builds the HTTP pipeline.

The seed data is six products. Two of them start at or below the low-stock threshold, so the low-stock endpoint and the warning colours in the frontend can be seen working on the very first run.

Chapter 3 — The Journey From Local Code to the Cloud

Stage 1 — Local code

The developer runs the app with the .NET SDK. The command **dotnet run** compiles the project, creates the SQLite database file, seeds it and starts listening. The frontend is at the root address and the API documentation is at the /swagger address. **Swagger is the fastest way to test every endpoint from the browser.**

Stage 2 — Docker

The Dockerfile builds the app in two stages. The first stage uses the full .NET SDK image and publishes a release build. The second stage uses the small ASP.NET Core runtime image and copies in only the published output. **The final image contains no source code and no SDK.** That makes it smaller and safer to ship. The command **docker build** produces the image and **docker run** starts it on port 8080.

Stage 3 — Docker Compose with PostgreSQL

The compose file starts two services: the app and a PostgreSQL 16 database with a health check and a named volume. Environment variables in the compose file set DatabaseProvider to Npgsql and supply the PostgreSQL connection string. **This stage is the proof of the configuration-only switch.** The image built in Stage 2 runs unchanged against a real database server.

Stage 4 — Kubernetes

The k8s folder holds five manifests. The **Deployment** runs two replicas with CPU and memory limits and with readiness and liveness probes against the products endpoint. The **Service** gives the pods one stable internal address. The **Ingress** routes public traffic for the host name to the Service. The **ConfigMap** carries the non-secret settings, including the DatabaseProvider value. The **Secret** carries the database connection string.

Trap: the Secret manifest in the repo contains placeholder values only. It exists to show the shape of the object. Real credentials are created directly in the cluster with the kubectl create secret command from

the README and are never written into a file that gets committed.

Stage 5 — Azure, AWS and CI/CD

On Azure the image is built in **Azure Container Registry** and run on **Azure Container Apps**, with **Azure Kubernetes Service** as the full-Kubernetes alternative. On AWS the image is pushed to **Amazon ECR** and run as an **ECS Fargate** service, with **Elastic Beanstalk** as the simple alternative. A GitHub Actions workflow builds and checks the project on every push, and on the main branch it pushes the image and updates both cloud deployments. **Cloud credentials live in GitHub secrets, never in the workflow file.** The workflow file names the secrets it needs and nothing more.

Chapter 4 — The Main Code Files in Full

Every file below is reproduced exactly as it exists in the project. Code keeps its own punctuation, including parentheses and comment markers.

Program.cs — startup, provider switch, seeding, pipeline

```
using InventoryHub.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// --- Services ------------------------------------------------------------
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// The database provider is chosen by CONFIGURATION ONLY.
//   Local run:      "DatabaseProvider": "Sqlite"   (appsettings.json default)
//   Docker / cloud: "DatabaseProvider": "Npgsql"   (environment variable)
// No code change is needed to move from SQLite to PostgreSQL.
var provider = builder.Configuration["DatabaseProvider"] ?? "Sqlite";
builder.Services.AddDbContext<AppDbContext>(options =>
{
    if (string.Equals(provider, "Npgsql", StringComparison.OrdinalIgnoreCase))
    {
        options.UseNpgsql(builder.Configuration.GetConnectionString("PostgresConnection"));
    }
    else
    {
        options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection"));
    }
});

var app = builder.Build();

// --- Database creation + seed data on startup (local SQLite profile) ------
// EnsureCreated builds the schema from the EF Core model when the database
// does not exist yet. For production, replace this with EF Core migrations.
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.EnsureCreated();
    if (!db.Products.Any())
    {
        db.Products.AddRange(AppDbContext.SeedProducts);
        db.SaveChanges();
    }
}

// --- HTTP pipeline -------------------------------------------------------
app.UseSwagger();
app.UseSwaggerUI();

app.UseDefaultFiles();   // serves wwwroot/index.html at "/"
app.UseStaticFiles();    // serves the single-page frontend from wwwroot

app.MapControllers();
app.MapFallbackToFile("index.html");

app.Run();
```

Models/Product.cs — the entity and its validation rules

```
using System.ComponentModel.DataAnnotations;
```

```
namespace InventoryHub.Models;

/// <summary>
/// One stocked product in the inventory.
/// Validation attributes are enforced automatically by [ApiController]:
/// an invalid request body returns HTTP 400 before the action runs.
/// </summary>
public class Product
{
    public int Id { get; set; }

    [Required]
    [StringLength(120, MinimumLength = 1)]
    public string Name { get; set; } = string.Empty;

    [Required]
    [StringLength(40, MinimumLength = 1)]
    public string Sku { get; set; } = string.Empty;

    [Required]
    [StringLength(60, MinimumLength = 1)]
    public string Category { get; set; } = string.Empty;

    [Range(0.01, 1_000_000)]
    public decimal Price { get; set; }

    [Range(0, 1_000_000)]
    public int StockQuantity { get; set; }

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

Data/AppDbContext.cs — EF Core context, unique index, seed data

```
using InventoryHub.Models;
using Microsoft.EntityFrameworkCore;

namespace InventoryHub.Data;

/// <summary>
/// Entity Framework Core database context.
/// The provider (SQLite locally, PostgreSQL in Docker/cloud) is wired up
/// in Program.cs from configuration, so this class stays provider-neutral.
/// </summary>
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {
    }

    public DbSet<Product> Products => Set<Product>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // A SKU identifies a product: enforce uniqueness at the database level.
        modelBuilder.Entity<Product>()
            .HasIndex(p => p.Sku)
            .IsUnique();

        modelBuilder.Entity<Product>()
            .Property(p => p.Price)
            .HasColumnType("decimal(18,2)");
    }

    /// <summary>Sample catalogue inserted on first run when the table is empty.</summary>
    public static Product[] SeedProducts => new[]
    {
        new Product { Id = 1, Name = "Wireless Mouse",      Sku = "WM-1001", Category = "Peripherals", Price = 24.99m,  StockQua
        new Product { Id = 2, Name = "Mechanical Keyboard", Sku = "KB-2001", Category = "Peripherals", Price = 89.50m,  StockQua
        new Product { Id = 3, Name = "27-inch Monitor",     Sku = "MN-2701", Category = "Displays",    Price = 219.99m, StockQua
```

```
        new Product { Id = 4, Name = "USB-C Docking Station", Sku = "DK-3001", Category = "Accessories", Price = 129.00m, StockQ
        new Product { Id = 5, Name = "Laptop Stand",        Sku = "LS-4001", Category = "Accessories", Price = 39.95m,  StockQua
        new Product { Id = 6, Name = "Noise-Cancelling Headphones", Sku = "HP-5001", Category = "Audio", Price = 149.99m, StockQ
    };
}
```

Controllers/ProductsController.cs — the full CRUD API

```
using InventoryHub.Data;
using InventoryHub.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace InventoryHub.Controllers;

/// <summary>
/// Full CRUD REST API for products under /api/products.
/// [ApiController] gives automatic model validation: invalid bodies
/// (missing name, negative price, ...) are rejected with HTTP 400.
/// </summary>
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly AppDbContext _db;

    public ProductsController(AppDbContext db) => _db = db;

    // GET /api/products
    [HttpGet]
    public async Task<ActionResult<IEnumerable<Product>>> GetAll()
        => await _db.Products.OrderBy(p => p.Id).ToListAsync();

    // GET /api/products/3
    [HttpGet("{id:int}")]
    public async Task<ActionResult<Product>> GetById(int id)
    {
        var product = await _db.Products.FindAsync(id);
        return product is null ? NotFound() : product;
    }

    // GET /api/products/low-stock?threshold=5
    // Products whose stock is below the threshold (default 5).
    [HttpGet("low-stock")]
    public async Task<ActionResult<IEnumerable<Product>>> GetLowStock([FromQuery] int threshold = 5)
        => await _db.Products
            .Where(p => p.StockQuantity < threshold)
            .OrderBy(p => p.StockQuantity)
            .ToListAsync();

    // POST /api/products
    [HttpPost]
    public async Task<ActionResult<Product>> Create(Product product)
    {
        if (await _db.Products.AnyAsync(p => p.Sku == product.Sku))
        {
            return Conflict(new { message = $"A product with SKU '{product.Sku}' already exists." });
        }

        product.Id = 0; // the database generates the id
        product.CreatedAt = DateTime.UtcNow;
        _db.Products.Add(product);
        await _db.SaveChangesAsync();

        return CreatedAtAction(nameof(GetById), new { id = product.Id }, product);
    }

    // PUT /api/products/3  (full update)
    [HttpPut("{id:int}")]
    public async Task<ActionResult<Product>> Update(int id, Product input)
```

```
    {
        var product = await _db.Products.FindAsync(id);
        if (product is null) return NotFound();

        product.Name = input.Name;
        product.Sku = input.Sku;
        product.Category = input.Category;
        product.Price = input.Price;
        product.StockQuantity = input.StockQuantity;
        await _db.SaveChangesAsync();

        return product;
    }

    // PATCH /api/products/3/stock  { "stockQuantity": 25 }
    [HttpPatch("{id:int}/stock")]
    public async Task<ActionResult<Product>> UpdateStock(int id, StockUpdateRequest request)
    {
        var product = await _db.Products.FindAsync(id);
        if (product is null) return NotFound();

        product.StockQuantity = request.StockQuantity;
        await _db.SaveChangesAsync();

        return product;
    }

    // DELETE /api/products/3
    [HttpDelete("{id:int}")]
    public async Task<IActionResult> Delete(int id)
    {
        var product = await _db.Products.FindAsync(id);
        if (product is null) return NotFound();

        _db.Products.Remove(product);
        await _db.SaveChangesAsync();

        return NoContent();
    }
}
```

Dockerfile — the two-stage container build

```
# InventoryHub — multi-stage Docker build
# Stage 1: build with the full .NET SDK
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Restore first (cached layer while only source files change)
COPY InventoryHub.csproj ./
RUN dotnet restore

COPY . ./
RUN dotnet publish -c Release -o /app/publish --no-restore

# Stage 2: small runtime image with only the ASP.NET Core runtime
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
COPY --from=build /app/publish ./

# Container port. In the cloud the provider injects its own PORT/hosting
# settings; locally and in Kubernetes the app listens on 8080.
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080

ENTRYPOINT ["dotnet", "InventoryHub.dll"]
```

docker-compose.yml — app plus PostgreSQL

```
# InventoryHub — full stack in Docker: app + PostgreSQL.
# The ONLY change from the local run is configuration: the app is switched
# from SQLite to PostgreSQL through environment variables.
#
#   docker compose up --build
#   App: http://localhost:8080

services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      # Configuration-only database switch (see Program.cs)
      DatabaseProvider: "Npgsql"
      ConnectionStrings__PostgresConnection: "Host=db;Port=5432;Database=inventoryhub;Username=postgres;Password=changeme"
      ASPNETCORE_URLS: "http://+:8080"
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: inventoryhub
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: changeme
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d inventoryhub"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  pgdata:
```

k8s/deployment.yaml — replicas, probes, config wiring

```
# InventoryHub — Kubernetes Deployment
# Runs 2 replicas of the app container. Configuration comes from the
# ConfigMap (non-secret settings) and the Secret (connection string).
apiVersion: apps/v1
kind: Deployment
metadata:
  name: inventoryhub
  labels:
    app: inventoryhub
spec:
  replicas: 2
  selector:
    matchLabels:
      app: inventoryhub
  template:
    metadata:
      labels:
        app: inventoryhub
    spec:
      containers:
        - name: inventoryhub
          # Replace with your registry image, e.g.
          # myregistry.azurecr.io/inventoryhub:1.0.0  or  <account>.dkr.ecr.<region>.amazonaws.com/inventoryhub:1.0.0
          image: inventoryhub:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          env:
```

```
            - name: ASPNETCORE_URLS
              value: "http://+:8080"
            - name: DatabaseProvider
              valueFrom:
                configMapKeyRef:
                  name: inventoryhub-config
                  key: DatabaseProvider
            - name: ConnectionStrings__PostgresConnection
              valueFrom:
                secretKeyRef:
                  name: inventoryhub-secret
                  key: PostgresConnection
          readinessProbe:
            httpGet:
              path: /api/products
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /api/products
              port: 8080
            initialDelaySeconds: 15
            periodSeconds: 20
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
```

.github/workflows/ci-cd.yml — build, image, Azure and AWS deploys

```
name: InventoryHub CI/CD

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

env:
  DOTNET_VERSION: '8.0.x'
  IMAGE_NAME: inventoryhub

jobs:
  # ---------------------------------------------------------------------
  # Build (and test, when a test project exists in the repo)
  # ---------------------------------------------------------------------
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: ${{ env.DOTNET_VERSION }}

      - name: Restore
        run: dotnet restore

      - name: Build
        run: dotnet build -c Release --no-restore

      - name: Test (only when test projects exist)
        shell: bash
        run: |
          TEST_PROJECTS=$(find . -name "*.Tests.csproj" -o -name "*Test*.csproj" | grep -v obj || true)
```

```
        if [ -n "$TEST_PROJECTS" ]; then
          dotnet test -c Release --no-build
        else
          echo "No test projects found — skipping test step."
        fi

    - name: Build Docker image
      run: docker build -t ${{ env.IMAGE_NAME }}:${{ github.sha }} .

# ------------------------------------------------------------------
# Deploy to Azure (Container Apps) — runs on pushes to main only.
# Required GitHub secrets:
#   AZURE_CREDENTIALS        (service principal JSON from `az ad sp create-for-rbac`)
#   ACR_LOGIN_SERVER         e.g. myregistry.azurecr.io
#   AZURE_RESOURCE_GROUP     e.g. inventoryhub-rg
#   AZURE_CONTAINERAPP_NAME  e.g. inventoryhub
# ------------------------------------------------------------------
deploy-azure:
  needs: build
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Log in to Azure
      uses: azure/login@v2
      with:
        creds: ${{ secrets.AZURE_CREDENTIALS }}

    - name: Build and push image to Azure Container Registry
      run: |
        az acr login --name ${{ secrets.ACR_LOGIN_SERVER }}
        docker build -t ${{ secrets.ACR_LOGIN_SERVER }}/${{ env.IMAGE_NAME }}:${{ github.sha }} .
        docker push ${{ secrets.ACR_LOGIN_SERVER }}/${{ env.IMAGE_NAME }}:${{ github.sha }}

    - name: Deploy to Azure Container Apps
      run: |
        az containerapp update \
          --name ${{ secrets.AZURE_CONTAINERAPP_NAME }} \
          --resource-group ${{ secrets.AZURE_RESOURCE_GROUP }} \
          --image ${{ secrets.ACR_LOGIN_SERVER }}/${{ env.IMAGE_NAME }}:${{ github.sha }}

# ------------------------------------------------------------------
# Deploy to AWS (ECS Fargate) — runs on pushes to main only.
# Required GitHub secrets:
#   AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_REGION
#   ECR_REPOSITORY           e.g. inventoryhub
#   ECS_CLUSTER, ECS_SERVICE e.g. inventoryhub-cluster / inventoryhub-service
# ------------------------------------------------------------------
deploy-aws:
  needs: build
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: ${{ secrets.AWS_REGION }}

    - name: Log in to Amazon ECR
      id: ecr
      uses: aws-actions/amazon-ecr-login@v2

    - name: Build and push image to ECR
      run: |
        IMAGE=${{ steps.ecr.outputs.registry }}/${{ secrets.ECR_REPOSITORY }}:${{ github.sha }}
```

```
          docker build -t $IMAGE .
          docker push $IMAGE
          echo "IMAGE=$IMAGE" >> $GITHUB_ENV

      - name: Deploy to ECS Fargate
        run: |
          aws ecs update-service \
            --cluster ${{ secrets.ECS_CLUSTER }} \
            --service ${{ secrets.ECS_SERVICE }} \
            --force-new-deployment
```

Chapter 5 — The Real Run Output

The text below is the captured output of the verification run, stored in the project as output.txt. The project was built in Release mode with zero warnings and zero errors. The compiled application was then started on port 5099 against a fresh database. The log shows Entity Framework Core creating the Products table and the unique SKU index, inserting the six seed products and starting the web host.

Read the API exchanges in order. The seed list returns six products. The POST creates a webcam as product 7 and returns HTTP 201. The low-stock call returns the three products below the threshold of 5, sorted by stock. The PATCH raises the laptop stand from 0 to 25 units. The deliberately invalid POST with a negative price returns HTTP 400, which proves the validation rules fire. The DELETE returns HTTP 204. The final call fetches the frontend page itself from the same running application.

```
======================================================================
InventoryHub — REAL RUN OUTPUT
Date: Fri Nov 13 00:03:48 UTC 2026
SDK: 8.0.428
======================================================================

--- Server startup log ---
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (89ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      PRAGMA journal_mode = 'wal';
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (0ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      CREATE TABLE "Products" (
          "Id" INTEGER NOT NULL CONSTRAINT "PK_Products" PRIMARY KEY AUTOINCREMENT,
          "Name" TEXT NOT NULL,
          "Sku" TEXT NOT NULL,
          "Category" TEXT NOT NULL,
          "Price" decimal(18,2) NOT NULL,
          "StockQuantity" INTEGER NOT NULL,
          "CreatedAt" TEXT NOT NULL
      );
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (0ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      CREATE UNIQUE INDEX "IX_Products_Sku" ON "Products" ("Sku");
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT EXISTS (
          SELECT 1
          FROM "Products" AS "p")
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (13ms) [Parameters=[@p0='?' (DbType = Int32), @p1='?' (Size = 11), @p2='?' (DbType = DateTime), @p3='?
      INSERT INTO "Products" ("Id", "Category", "CreatedAt", "Name", "Price", "Sku", "StockQuantity")
      VALUES (@p0, @p1, @p2, @p3, @p4, @p5, @p6);
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (0ms) [Parameters=[@p0='?' (DbType = Int32), @p1='?' (Size = 11), @p2='?' (DbType = DateTime), @p3='?'
      INSERT INTO "Products" ("Id", "Category", "CreatedAt", "Name", "Price", "Sku", "StockQuantity")
      VALUES (@p0, @p1, @p2, @p3, @p4, @p5, @p6);
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (0ms) [Parameters=[@p0='?' (DbType = Int32), @p1='?' (Size = 8), @p2='?' (DbType = DateTime), @p3='?' (
      INSERT INTO "Products" ("Id", "Category", "CreatedAt", "Name", "Price", "Sku", "StockQuantity")
      VALUES (@p0, @p1, @p2, @p3, @p4, @p5, @p6);
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (0ms) [Parameters=[@p0='?' (DbType = Int32), @p1='?' (Size = 11), @p2='?' (DbType = DateTime), @p3='?'
      INSERT INTO "Products" ("Id", "Category", "CreatedAt", "Name", "Price", "Sku", "StockQuantity")
      VALUES (@p0, @p1, @p2, @p3, @p4, @p5, @p6);
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (0ms) [Parameters=[@p0='?' (DbType = Int32), @p1='?' (Size = 11), @p2='?' (DbType = DateTime), @p3='?'
      INSERT INTO "Products" ("Id", "Category", "CreatedAt", "Name", "Price", "Sku", "StockQuantity")
      VALUES (@p0, @p1, @p2, @p3, @p4, @p5, @p6);
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (0ms) [Parameters=[@p0='?' (DbType = Int32), @p1='?' (Size = 5), @p2='?' (DbType = DateTime), @p3='?' (
```

```
      INSERT INTO "Products" ("Id", "Category", "CreatedAt", "Name", "Price", "Sku", "StockQuantity")
      VALUES (@p0, @p1, @p2, @p3, @p4, @p5, @p6);
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://127.0.0.1:5099
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /home/hatch/workspace/inventoryhub-dotnet
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT "p"."Id", "p"."Category", "p"."CreatedAt", "p"."Name", "p"."Price", "p"."Sku", "p"."StockQuantity"
      FROM "Products" AS "p"
      ORDER BY "p"."Id"

--- GET /api/products (seed data) ---
[
    {
        "id": 1,
        "name": "Wireless Mouse",
        "sku": "WM-1001",
        "category": "Peripherals",
        "price": 24.99,
        "stockQuantity": 140,
        "createdAt": "2026-01-05T09:00:00"
    },
    {
        "id": 2,
        "name": "Mechanical Keyboard",
        "sku": "KB-2001",
        "category": "Peripherals",
        "price": 89.5,
        "stockQuantity": 64,
        "createdAt": "2026-01-05T09:00:00"
    },
    {
        "id": 3,
        "name": "27-inch Monitor",
        "sku": "MN-2701",
        "category": "Displays",
        "price": 219.99,
        "stockQuantity": 23,
        "createdAt": "2026-01-06T09:00:00"
    },
    {
        "id": 4,
        "name": "USB-C Docking Station",
        "sku": "DK-3001",
        "category": "Accessories",
        "price": 129,
        "stockQuantity": 4,
        "createdAt": "2026-01-06T09:00:00"
    },
    {
        "id": 5,
        "name": "Laptop Stand",
        "sku": "LS-4001",
        "category": "Accessories",
        "price": 39.95,
        "stockQuantity": 0,
        "createdAt": "2026-01-07T09:00:00"
    },
    {
        "id": 6,
        "name": "Noise-Cancelling Headphones",
        "sku": "HP-5001",
        "category": "Audio",
        "price": 149.99,
        "stockQuantity": 37,
```

```
        "createdAt": "2026-01-07T09:00:00"
    }
]

--- POST /api/products (create) ---
Request body: {"name":"Webcam HD 1080p","sku":"WC-6001","category":"Peripherals","price":59.99,"stockQuantity":3}
{"id":7,"name":"Webcam HD 1080p","sku":"WC-6001","category":"Peripherals","price":59.99,"stockQuantity":3,"createdAt":"2026-11-1
HTTP status: 201


--- GET /api/products/low-stock?threshold=5 ---
[
    {
        "id": 5,
        "name": "Laptop Stand",
        "sku": "LS-4001",
        "category": "Accessories",
        "price": 39.95,
        "stockQuantity": 0,
        "createdAt": "2026-01-07T09:00:00"
    },
    {
        "id": 7,
        "name": "Webcam HD 1080p",
        "sku": "WC-6001",
        "category": "Peripherals",
        "price": 59.99,
        "stockQuantity": 3,
        "createdAt": "2026-11-13T00:03:48.764988"
    },
    {
        "id": 4,
        "name": "USB-C Docking Station",
        "sku": "DK-3001",
        "category": "Accessories",
        "price": 129,
        "stockQuantity": 4,
        "createdAt": "2026-01-06T09:00:00"
    }
]

--- PATCH /api/products/5/stock {stockQuantity: 25} ---
{
    "id": 5,
    "name": "Laptop Stand",
    "sku": "LS-4001",
    "category": "Accessories",
    "price": 39.95,
    "stockQuantity": 25,
    "createdAt": "2026-01-07T09:00:00"
}

--- POST invalid product (validation check: negative price) ---
HTTP status: 400

--- DELETE /api/products/7 ---
HTTP status: 204

--- GET / (frontend index page, first 12 lines) ---
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>InventoryHub — Product Inventory Manager</title>
<style>
  :root { --blue:#1f6feb; --dark:#0d2137; --light:#f4f7fb; --red:#c0392b; --amber:#b9770e; }
  * { box-sizing: border-box; }
  body { margin:0; font-family: "Segoe UI", Arial, sans-serif; background: var(--light); color:#222; }
  header { background: var(--dark); color:#fff; padding: 18px 28px; }
```

```
  header h1 { margin:0; font-size: 24px; }
```

Every check passed. Create, read, update, delete, low-stock filtering, validation and frontend serving were all exercised against the running application, not merely compiled.

Chapter 6 — Deploying to Azure

The full guide lives in azure-deploy.md in the project root. The short version has four moves. Create a resource group and a container registry. Build the image inside the registry with the az acr build command, which means no local Docker installation is required. Create the container app with the registry image, target port 8080 and the database settings passed as environment variables. Read back the public address of the app.

```
az group create --name inventoryhub-rg --location eastus
az acr create --resource-group inventoryhub-rg --name <registry> --sku Basic
az acr build --registry <registry> --image inventoryhub:1.0.0 .
az containerapp create --name inventoryhub --resource-group inventoryhub-rg \
  --environment inventoryhub-env --image <registry>.azurecr.io/inventoryhub:1.0.0 \
  --target-port 8080 --ingress external --registry-server <registry>.azurecr.io \
  --env-vars DatabaseProvider=Npgsql "ConnectionStrings__PostgresConnection=<secret>"
```

The AKS path reuses the Kubernetes manifests from Chapter 4. The only edits are the image name, which points at the registry, and the Secret, which is created in the cluster. **Point the Deployment at your registry image before applying, or the pods will try to pull an image that does not exist.**

For continuous deployment, the workflow needs a service principal. The azure-deploy.md file shows the az ad sp create-for-rbac command and the four GitHub secrets the deploy job reads.

Chapter 7 — Deploying to AWS

The full guide lives in aws/aws-deploy.md. The short version mirrors the Azure path. Create an ECR repository and push the image. Store the PostgreSQL connection string in AWS Secrets Manager. Register the task definition from aws/ecs-task-definition.json after replacing the account, region and role placeholders. Create the Fargate service behind an Application Load Balancer.

```
aws ecr create-repository --repository-name inventoryhub
aws ecr get-login-password | docker login --username AWS --password-stdin <account>.dkr.ecr.<region>.amazonaws.com
docker build -t inventoryhub:1.0.0 .
docker tag inventoryhub:1.0.0 <account>.dkr.ecr.<region>.amazonaws.com/inventoryhub:1.0.0
docker push <account>.dkr.ecr.<region>.amazonaws.com/inventoryhub:1.0.0
aws ecs register-task-definition --cli-input-json file://aws/ecs-task-definition.json
aws ecs create-service --cluster inventoryhub-cluster --service-name inventoryhub-service \
  --task-definition inventoryhub --desired-count 2 --launch-type FARGATE
```

Trap: the task definition reads the connection string from Secrets Manager. The ECS task execution role must be allowed to read that secret, or the task stops before the container starts. The Elastic Beanstalk path in aws/aws-deploy.md skips containers entirely and publishes the compiled application as a zip. **Beanstalk is the right choice for a quick single-instance demo, and Fargate is the right choice for the production shape.**

Chapter 8 — Limitations, Stated Honestly

- **Cloud deploys are configuration-ready, not executed.** Every Azure and AWS command in this project follows the real CLI syntax, and the same container and manifests are used in the verified local and Docker paths. No live Azure subscription or AWS account was used while building the project, so the cloud steps have not been run end to end.
- **Database creation uses EnsureCreated.** That is the right tool for a zero-setup local profile and a demo. A production system should generate EF Core migrations and apply them as a controlled deployment step.
- **There is no authentication.** Anyone who can reach the API can change the inventory. Add ASP.NET Core Identity or JWT bearer authentication before exposing the application publicly.
- **The verification environment fought back twice.** The SDK installer reported an extraction error on a full system temp area, and the first NuGet restore timed out through the network proxy. Both were worked around, the retry succeeded, and the final build finished with zero warnings and zero errors. The run output in Chapter 5 is from that final build.
- **The repository ships no test project.** The CI workflow detects test projects automatically and runs them when they exist. Adding an xUnit project later needs no workflow change.

Glossary

API — the surface an application exposes so other programs can call it. Here it is a set of HTTP routes returning JSON.

REST — a style of API design that maps operations onto HTTP methods and resource addresses.

CRUD — create, read, update and delete. The four basic operations on stored data.

Entity Framework Core — the object-relational mapper that turns C# objects and queries into database tables and SQL.

DbContext — the Entity Framework Core class that represents a session with the database and exposes the entity sets.

Migration — a versioned, reviewable description of a database schema change, generated from the model.

Seed data — starter rows inserted into an empty database so the application is useful on first run.

SKU — stock keeping unit. The short code that identifies one product in a catalogue.

Swagger — a generated web page that documents the API and lets a developer call endpoints from the browser.

Docker image — a packaged, immutable snapshot of an application and its runtime, ready to start as a container.

Docker Compose — a tool that runs several related containers from one YAML file.

Kubernetes — a system that runs containers across a cluster of machines and keeps the desired number of copies alive.

Deployment — the Kubernetes object that declares how many replicas of a container should run and which image they use.

Service — the Kubernetes object that gives a set of pods one stable network address.

Ingress — the Kubernetes object that routes public HTTP traffic from a host name to a Service.

ConfigMap — a Kubernetes object holding non-secret configuration as key and value pairs.

Secret — a Kubernetes object holding sensitive configuration such as a connection string.

Azure Container Registry — the Azure service that stores private Docker images.

Azure Container Apps — the Azure service that runs containers without managing servers or a cluster.

AKS — Azure Kubernetes Service. The managed Kubernetes offering on Azure.

ECR — Amazon Elastic Container Registry. The AWS service that stores private Docker images.

ECS Fargate — Amazon Elastic Container Service running tasks on Fargate, the serverless container engine on AWS.

Elastic Beanstalk — the AWS service that deploys a web application from uploaded code with minimal setup.

CI/CD — continuous integration and continuous deployment. Automated building, checking and releasing on every change.

End of the explainer. The project files, the README and the captured run output live together in the InventoryHub repository.